Thread-local variables share other properties with static variables—they're zero-initialized prior to any further initialization (such as dynamic initialization), and if the construction of a thread-local variable throws an exception, `std::terminate()` is called to abort the application.

The destructors for all thread-local variables that have been constructed on a given thread are run when the thread function returns, in the reverse order of construction. Since the order of initialization is unspecified, it's important to ensure that there are no interdependencies between the destructors of such variables. If the destructor of a thread-local variable exits with an exception, `std::terminate()` is called, just as for construction.

Thread-local variables are also destroyed for a thread if *that thread* calls `std::exit()` or returns from main() (which is equivalent to calling `std::exit()` with the return value of `main()`). If any other threads are still running when the application exits, the destructors of thread-local variables on those threads are *not* called.

Though thread-local variables have a different address on each thread, you can still obtain a normal pointer to such a variable. The pointer then references the object in the thread that took the address, and can be used to allow other threads to access that object. It's undefined behaviour to access an object after it's been destroyed (as always), so if you pass a pointer to a thread-local variable to another thread, you need to ensure it's not dereferenced once the owning thread has finished.

A.9. Summary

This appendix has only scratched the surface of the new language features introduced with the C++11 Standard, because we've only looked at those features that actively affect the usage of the Thread Library. Other new language features include static assertions, strongly typed enumerations, delegating constructors, Unicode support, template aliases, and a new uniform initialization sequence, along with a host of smaller changes. Describing all the new features in detail is outside the scope of this book; it would probably require a book in itself. The best overview of the entire set of